
포트폴리오

(그래픽스 엔진)



2026. 05. 5
권영일

Polyhedron

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Attributes

- Vertices. Faces
- Bounding Box, Center Point, Centeroid

Edition

- Smoothing
- Deformation
- Compound
- Explode
- Labeling
- Divide by Polygon to Cut
- Extrude Partial Part
- Transformation
- Slices (Sections) By Z-Axis
- Add/Remove Triangles

Weight and Measures

- Volume
- Weight

Transformation

- Vector
- Matrix3x3, Matrix2x4

Creation

- (Arc) Arrow
- Cylinder
- Grid
- From 2D Polygon/Polygons
- Solid (thickness) from Polygons

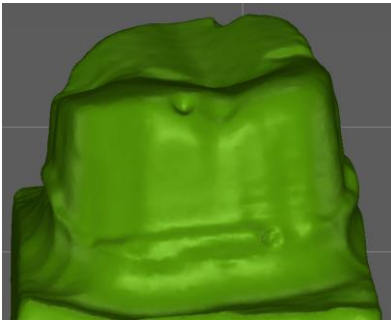
Polyhedron(Continue)

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Creation

- (Arc) Arrow
- Cylinder
- Grid
- From 2D Polygon/Polygons
- Solid (thickness) from Polygons

Shape & Code



```
CGPolyhedron * iPolyhedron = new CGPolyhedron;  
VERIFY(! (ret = iPolyhedron->ReadSTL(iFilePath, NULL)));
```

Best Fit

- ICP Registration

Projection

- Project Point Onto Mesh along a Direction
- Project Polygons Onto Mesh along a Direction

Fix

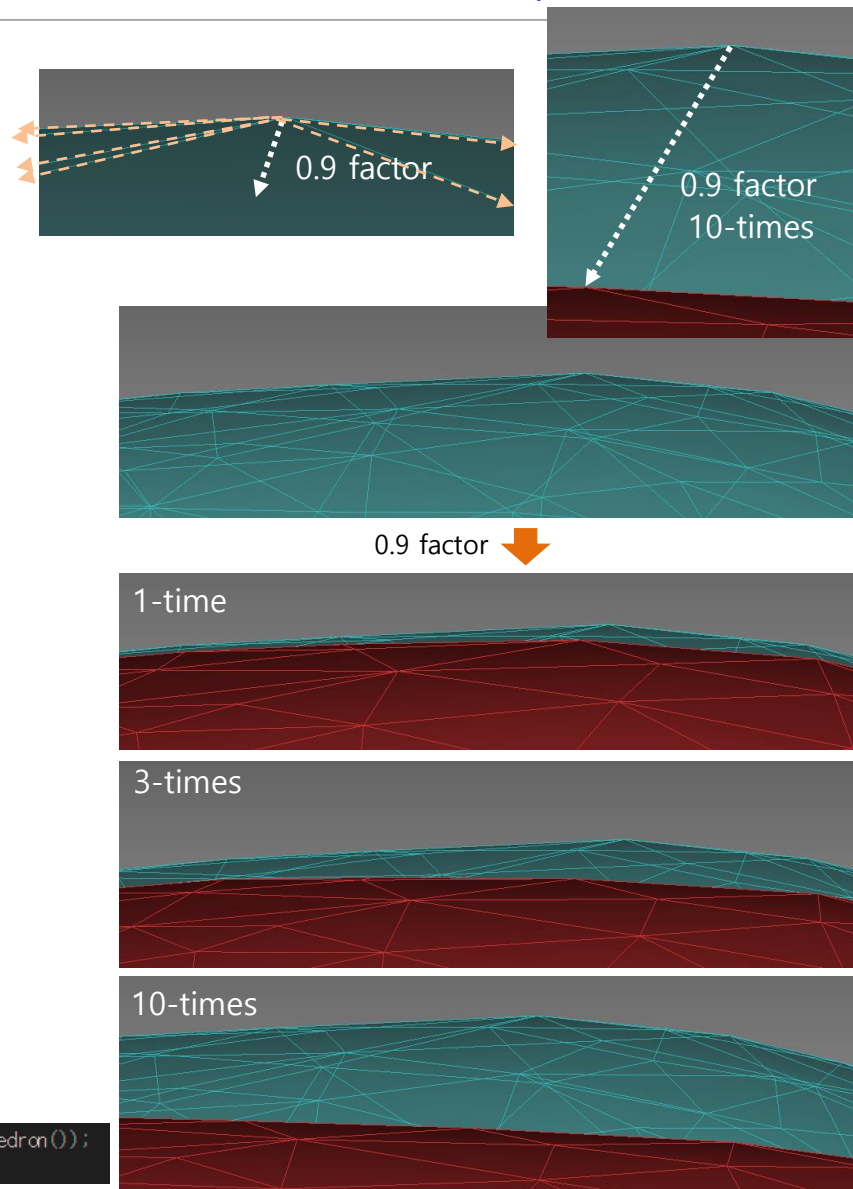
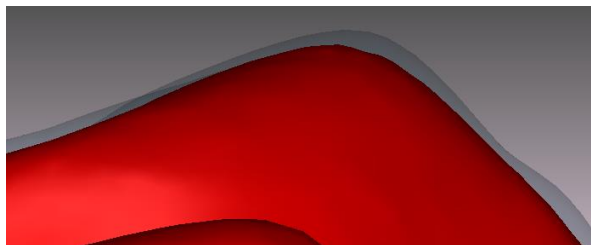
- Remove Duplicated Faces
- Fix Face Normals
- Invert Normals

Polyhedron : Edition

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Smoothing

- Entire Smoothing : 전체 Vertex에 대한 Vertex Smoothing 수행
- Partial Smoothing : 일부 영역의 Vertices를 Vertex Smoothing 처리
- Vertex Smoothing : 인접된 Face의 Edge 방향을 이용한 Smoothing

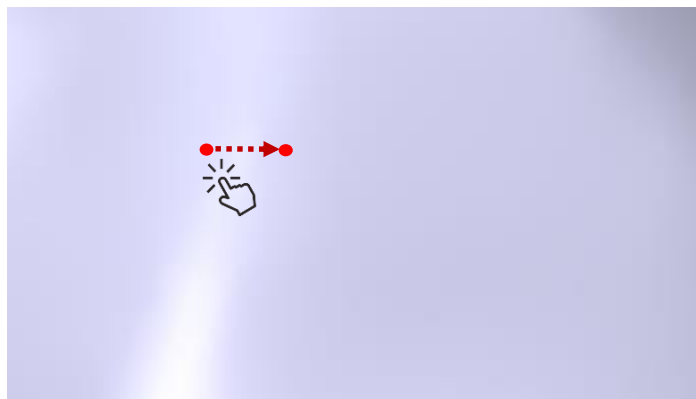


```
CGPolyhedron * iPolyhedron = const_cast<CGPolyhedron *>((const CGPolyhedron *)m_DPolyhedronSmooth.GetPolyhedron());
VERIFY(iPolyhedron->Smooth(.9));
```

Polyhedron : Edition

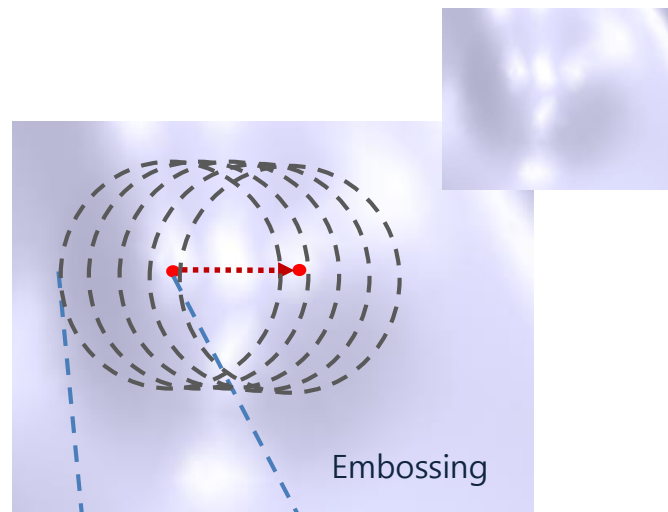
[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Deformation

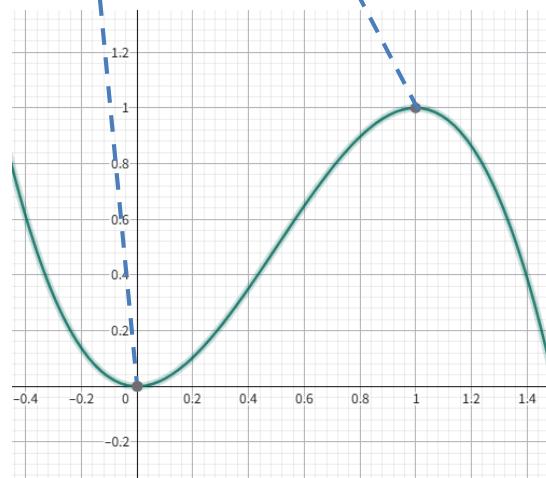


Mouse Click and Drag

```
CStretch stretch(const_cast<CGPolyhedron*>(&*m_PolyToDeform->GetPolyhedron()), pt, 1.6);
stretch.MoveDirection(m_nDeformType?viewDirection:-viewDirection);
stretch.MoveDistance(.03);
```



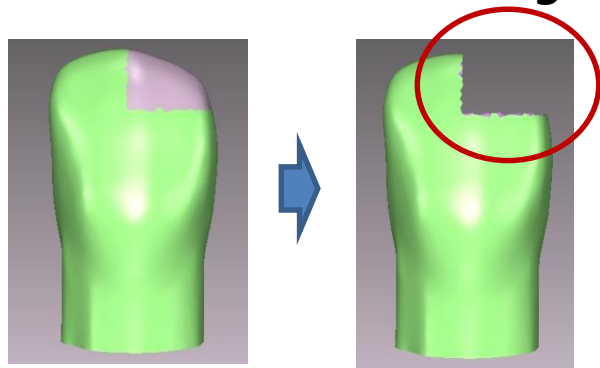
Cubic Equation : $3x^2 - 2x^3$



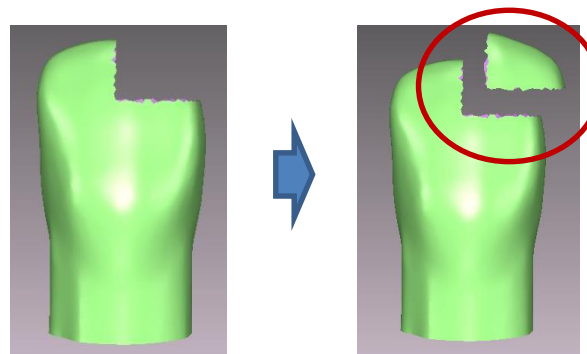
Polyhedron : Edition

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Addition/Deletion Triangles

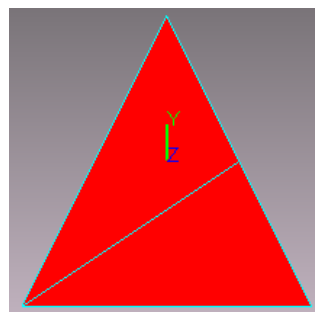
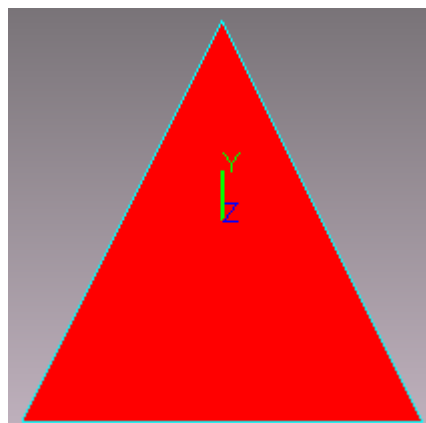


Deletion

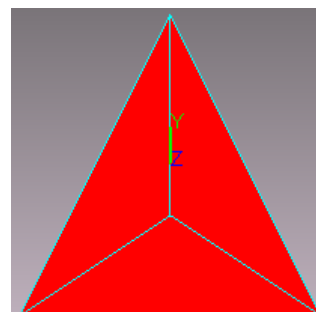


Addition

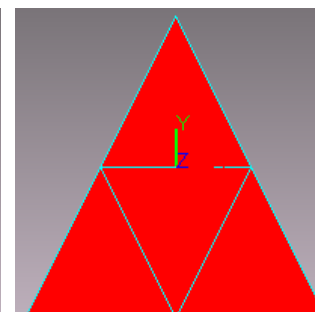
Subdivision the Triangle



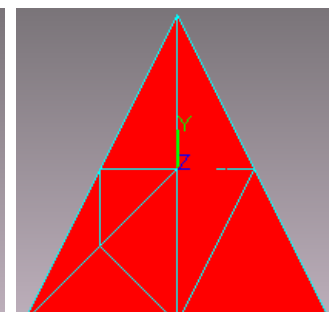
2 분할



3 분할



4분할



길이 분할
(6으로 분할)

Polyhedron : Compound & Explode

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

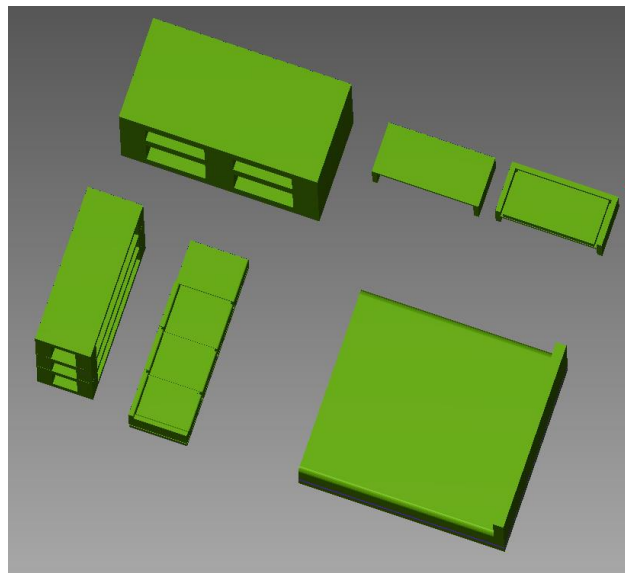
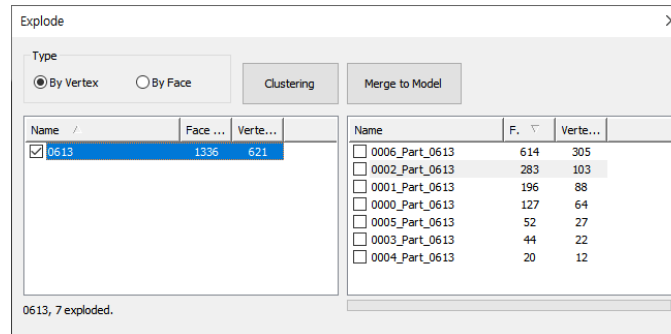
Compound

: Vertices, Faces 모두 병합

Explode(Segmentation)

: 2가지 방식

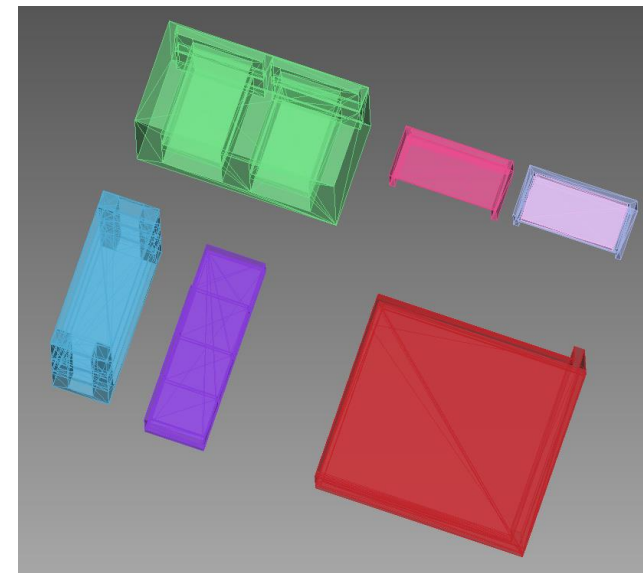
- By Vertex
- By Face



Explode



Compound

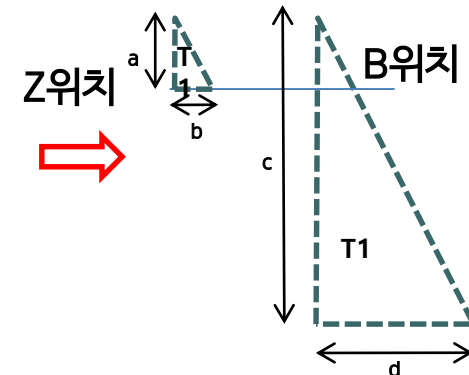
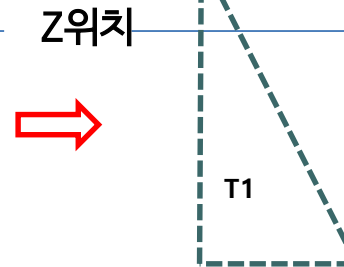
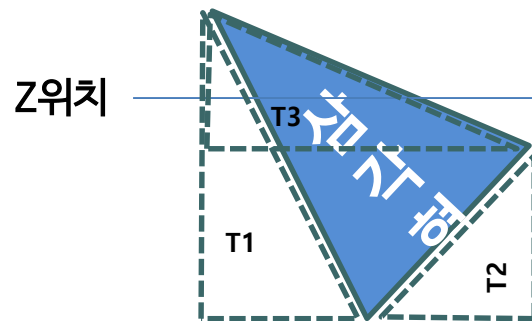


Polyhedron : Slicing

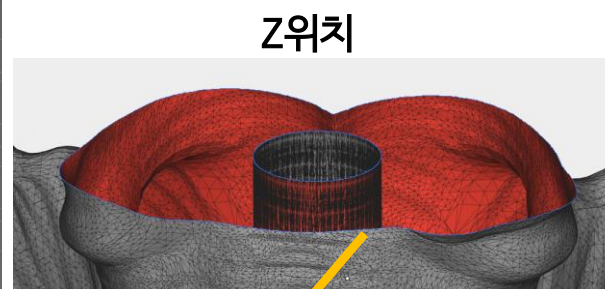
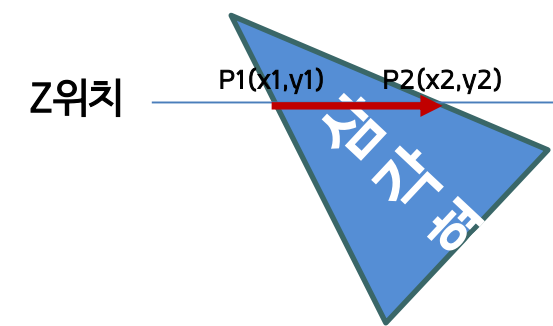
[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Slice Section by Z-Axis

: Z축 기준으로 0.5~2mm 간격으로 Sliced Polygons 생성



$$a : c = b : d$$

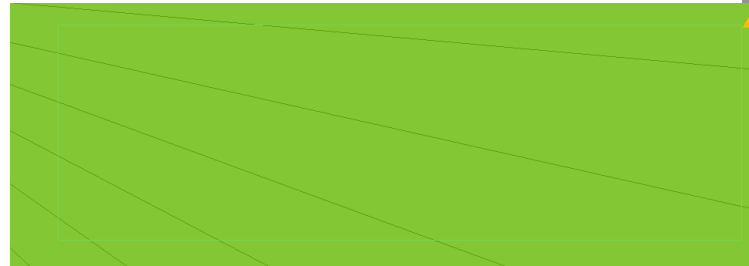
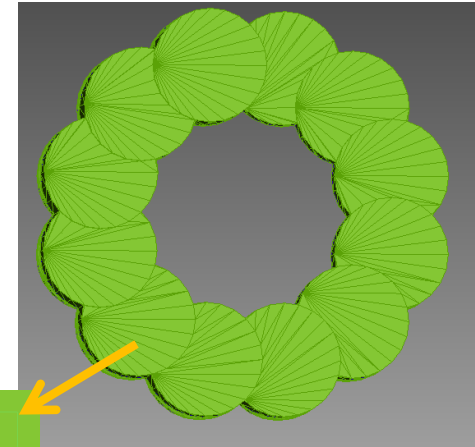
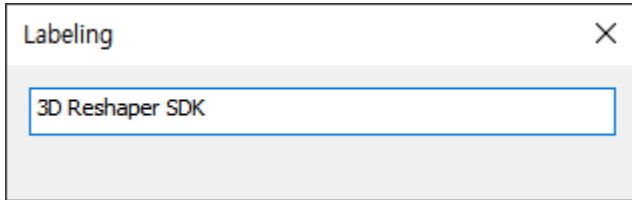


Sliced Polygons

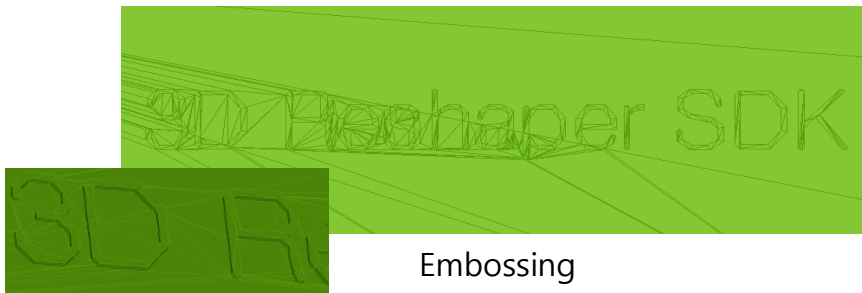
Polyhedron : Labeling

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Labeling On Mesh



Projection & Labeling



Embossing

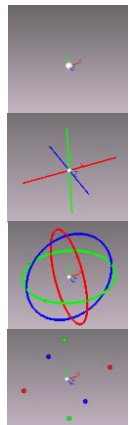


Debossing

Mesh : Transformation

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Transformation Using Manipulator 1

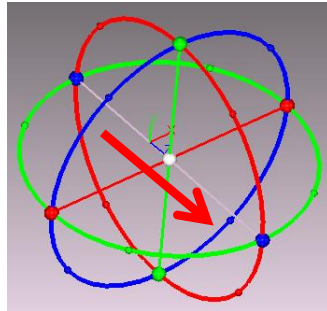


Center

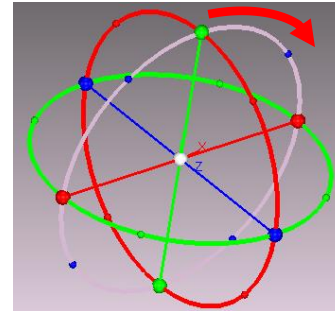
Translate

Rotate

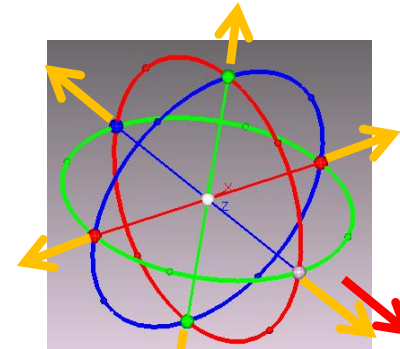
Scale



가운데 3축을 이용하여
Translate를 실행한다.

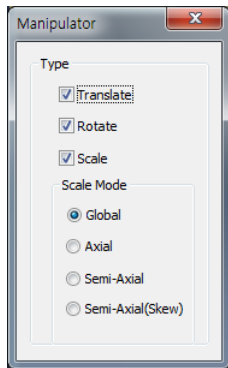


Circle 3축을 이용하여 Rotate
를 실행한다.

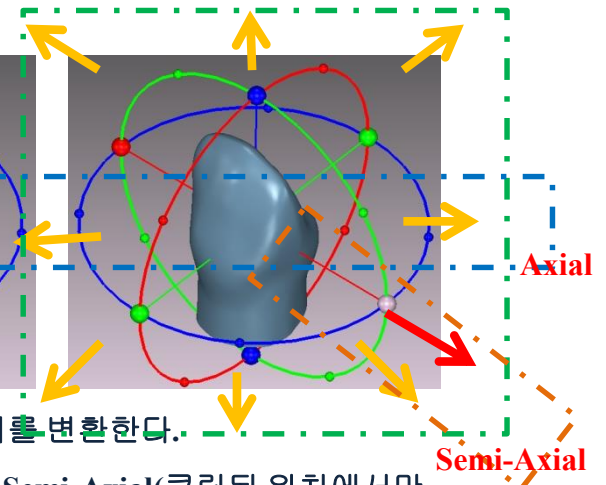
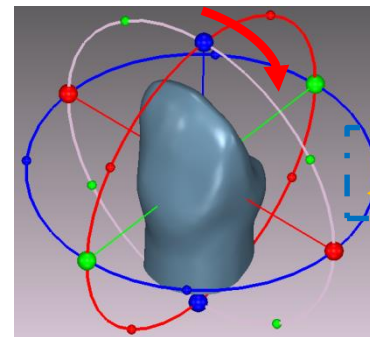
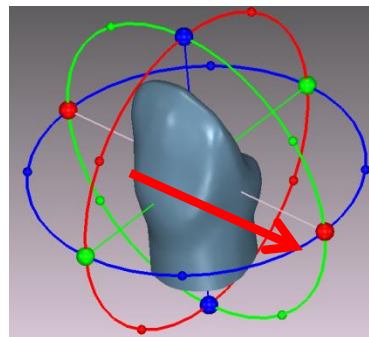


Translate 축 끝 점을 이용하여
Scale을 실행한다.

Mesh Transformation



변환할 타입을 선택하여 실행한다.



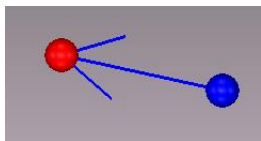
마우스를 이용하여 이동, 회전, 크기를 변환한다.

Global(x,y,z축 크기변환), Semi(클릭된 축 크기변환), Semi-Axial(클릭된 위치에서만 크기 변환), Semi-Axial(기준점이 Polyhedron 반대편을 기준으로 크기변환)

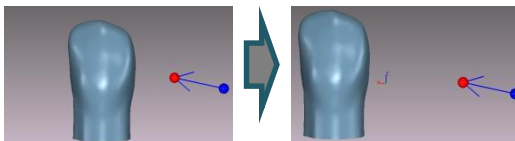
Polyhedron : Transformation

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Translate the mesh

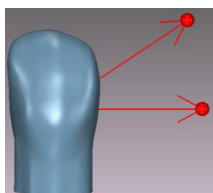


Scene에 클릭하여 이동할 방향을 설정한다.(취소는 DEL키)



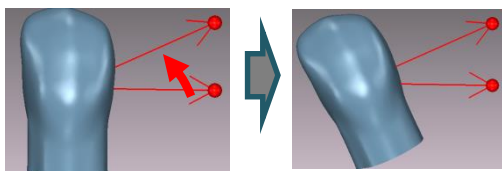
Execute 버튼을 이용하여 이동한다.

Rotation the mesh about arbitrary

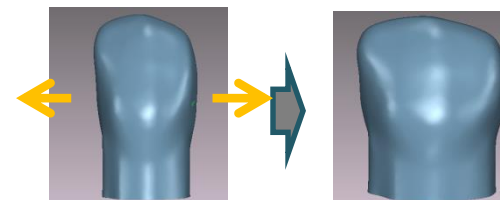


Scene에 클릭하여 이동할 방향을 설정한다.(삭제는 DEL키)

View Direction 축으로 회전

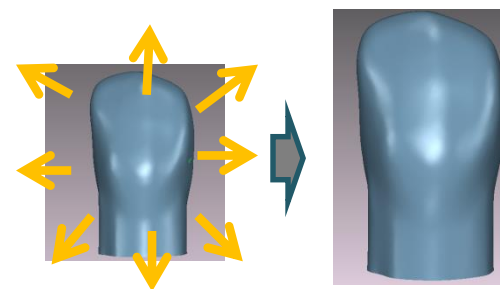


Scale the mesh



X축으로 Scale factor 1.5를 주고 실행

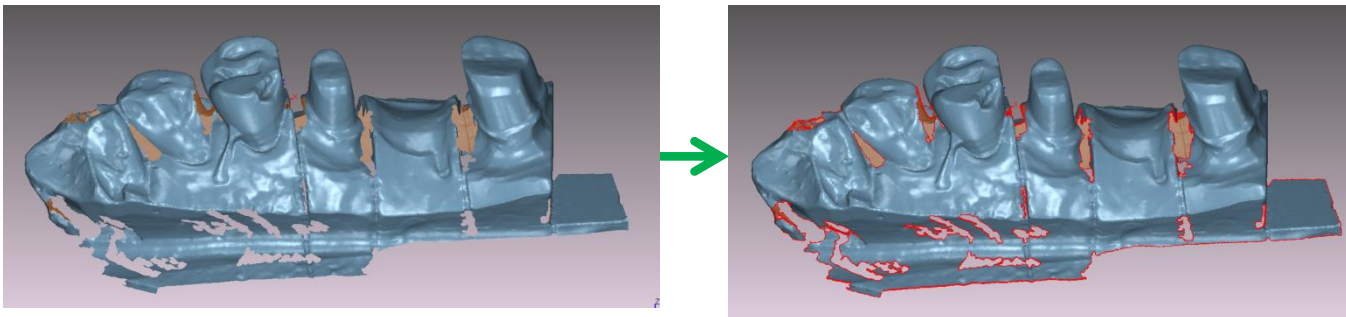
Polyhedron 중심점을 기준으로 크기 변환



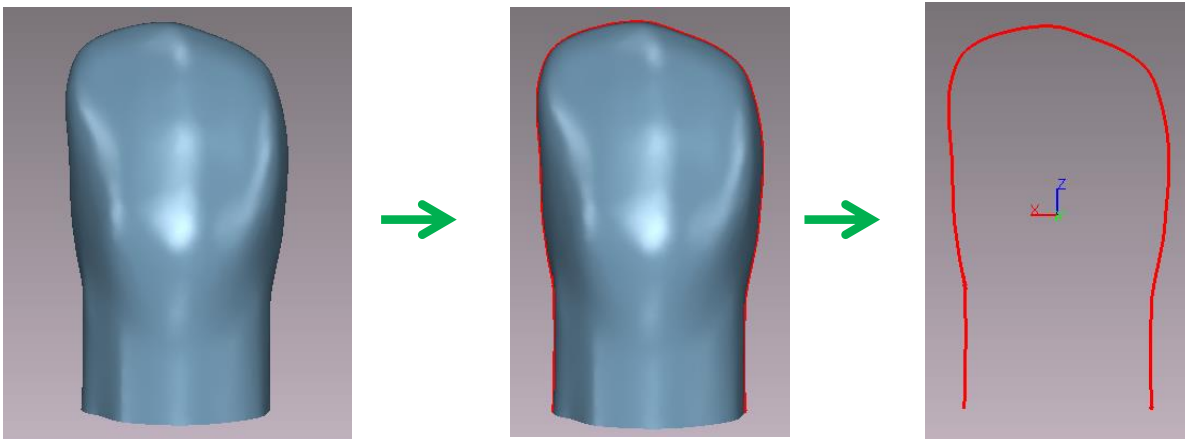
Polyhedron : Finding

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Find the free-border in Mesh



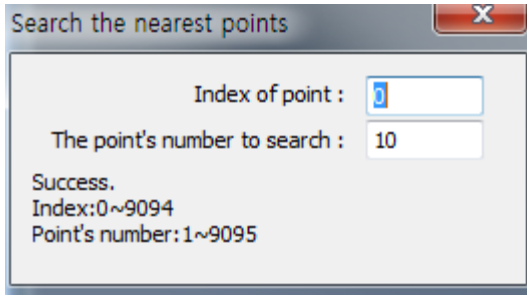
Silhouette



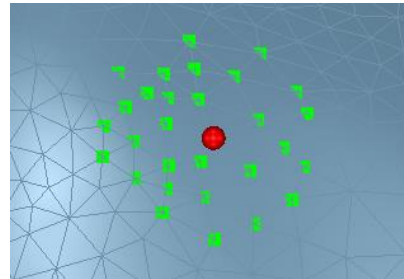
Polyhedron : Finding

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

Find the Nearest Points In Mesh



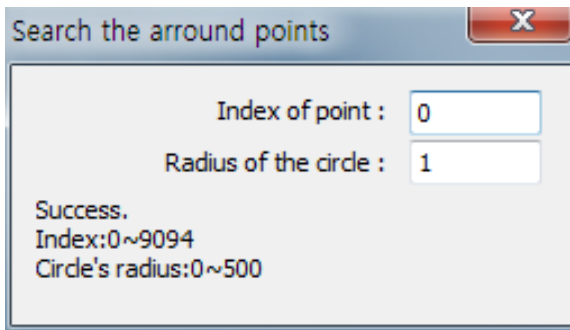
Point Index를 기준으로 Point Number 기입한다.



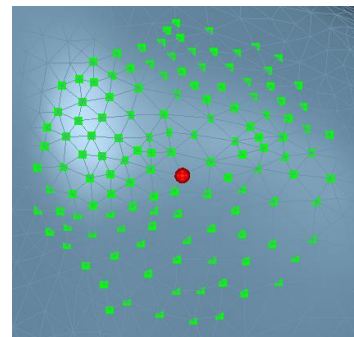
한 점에서 정확히 가장 가까운 점을 찾았는지 확인한다.

주의) 빨간색 점 위치도 찾은 점이 표시되었지만 빨간색 점 때문에 보이지 않는다.

Find around Points In Mesh



Point Index를 기준으로 Circle's Radius 기입한다.



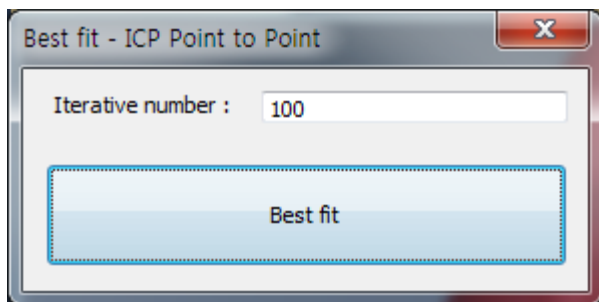
한 점에서 정확히 원형으로 찾는지 확인한다.

주의) 빨간색 점 위치도 찾은 점이 표시되었지만 빨간색 점 때문에 보이지 않는다.

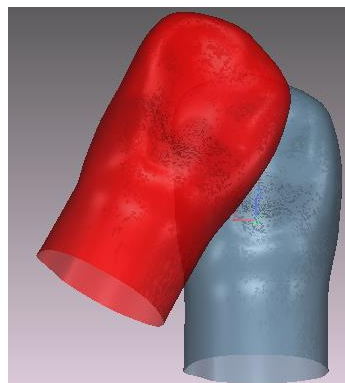
Polyhedron : Best Fit

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/Mesh/GPolyhedron.h](#)

ICP : Point to Point

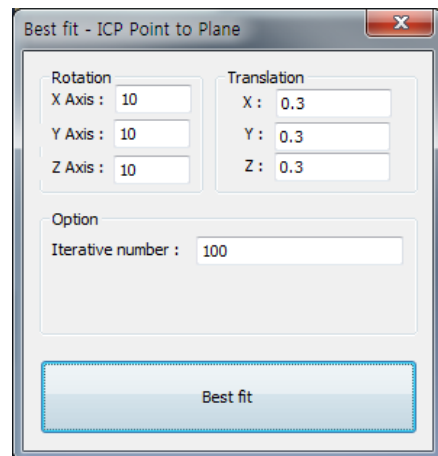


Best fit버튼을 눌러 Align을 실행한다.

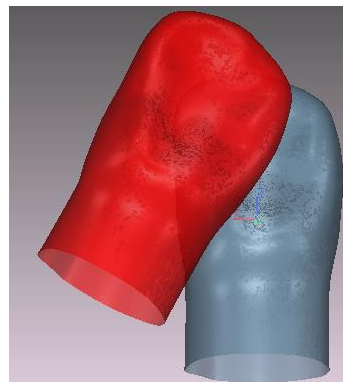


Align이 잘 되었는지 확인한다.

ICP : Point to Plane



회전 및 이동을 시킨 후 Best fit을 누른다.



Align이 잘 되었는지 확인한다.

Point

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPointDouble.h](#)

Attributes

- X, Y

Edition

- Invert

Retrieve

- Retrieve Accuracy value

Projection

- Project a Point on Line along Y-Axis
- Project a Point on Sphere
- Project a Point on Plane along a Direction

Weight & Measures

- Length, Square Length

Transformation

- Vector2D, Vector
- Matrix2x2, Matrix2x3, Matrix3x3, Matrix3x4

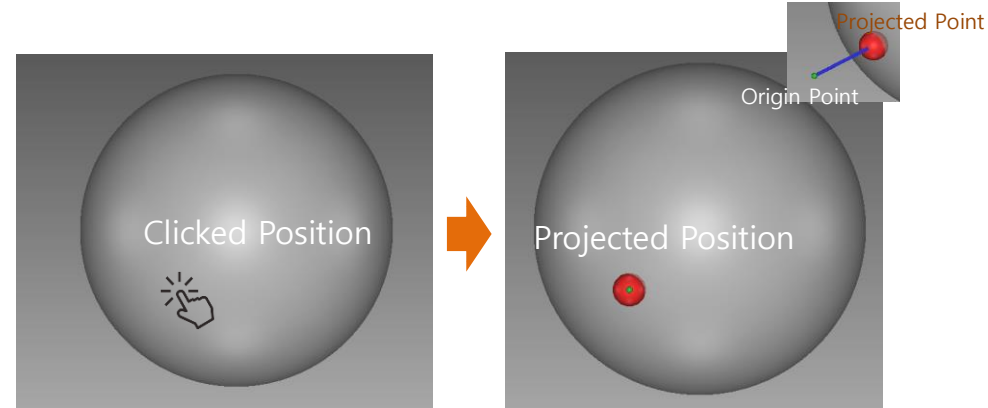
Point : Projection

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPointDouble.h](#)

Project a Point on Sphere

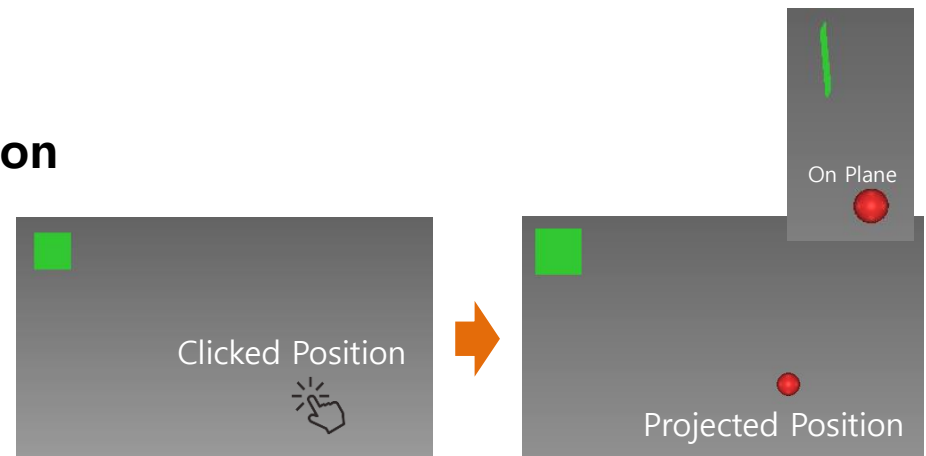
- 클릭된 위치에서 뷰 방향으로 구의 표면에 투영된다.

```
CGPointDouble result;
if (iSpherePoint.Project(iDeparturePoint, iViewDirection, m_DSphere.GetPointSize() * .5, &result) == 1)
{
    iArrivalPoint = result;
    iSucceed = TRUE;
}
```



Project a Point on Plane along a Direction

```
m_DPoint.SetPoint(iClickedPoint);
iClickedPoint.ProjectToPlane(iViewDirection, m_Plane.Interpolate(0, 0));
```



2D Point

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPoint2DDouble.h](#)

Attributes

- X, Y

Edition

- Invert

Retrieve

- Is Contain X/Y In X/Y Range
- Is Left Position of Line
- Retrieve Accuracy value

Projection

- Project a Point on Line along Y-Axis

Weight & Measures

- Length, Square Length

Transformation

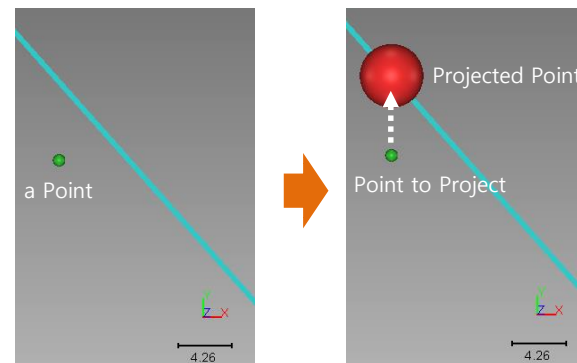
- Vector2D
- Matrix3x3, Matrix3x4

2D Point : Projection

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPointDouble.h](#)

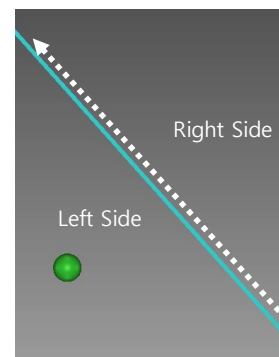
Project a Point on Line along Y-Axis

```
CGPoint2DDouble iPointToProject(m_DPoint.GetPoint());
CGPoint2DDouble iProjectedPoint = CGPoint2DDouble::ProjectOnLineYAxis(iPointToProject.ToCoordinate2D(),
                                                                    m_DLine.GetLine().First().ToCoordinate2D(),
                                                                    m_DLine.GetLine().Last().ToCoordinate2D());
m_DProjectedPoint.SetPoint(iProjectedPoint);
m_DProjectedPoint.Show();
```



Is Left Position of Line

```
if (CGPoint2DDouble::IsLeft(m_DLine.GetLine().First(), m_DLine.GetLine().Last(), iPointToProject) >= 0.)
    TRACE(_T("Point In Left Side"));
else
    TRACE(_T("Point In Right Side"));
```



Polygon

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygon.h](#)

Attributes

- Vertices
- Bounding Box, Center Point, Centeroid

Edition

- Add/Remove Points
- Add Polygon
- Open/Close
- Change Starting Point

Projection

- Project Polygons Onto Mesh along a Direction
- Project Polygons Onto Mesh along Z-Axis

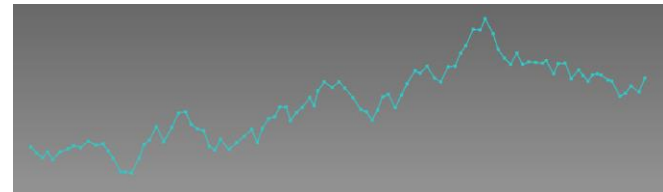
Weight & Measures

- Length

Transformation

- Vector, Vector2D
- Matrix2x2, Matrix2x3, Matrix3x3, Matrix2x4

Shape & Code



```
CGPolygon iPolygon;
CGPolygon::_VertexVector & iPoints = iPolygon.Vertices();
iPoints.resize(100);
Coordinate3D iPreCoordinate(50, 50, 50);
for (UINT ii = 0; ii < (UINT)iPoints.size(); ii++)
{
    Coordinate3D & iCoordinate = iPoints[ii];

    iCoordinate._xx = iPreCoordinate._xx + CUtil::Random(0.5, 1.);
    iCoordinate._yy = iPreCoordinate._yy + CUtil::Random(-2., 2.);
    iCoordinate._zz = iPreCoordinate._zz + CUtil::Random(-2., 2.);

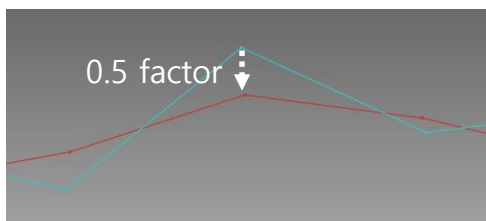
    iPreCoordinate = iCoordinate;
}
```

Polygon : Edition

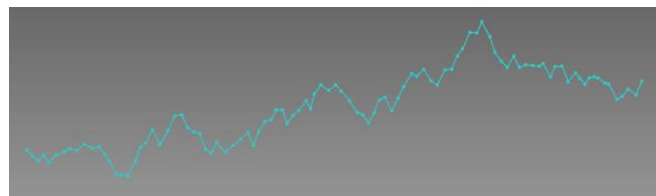
[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygon.h](#)

Smoothing

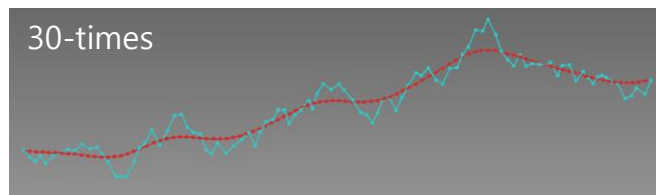
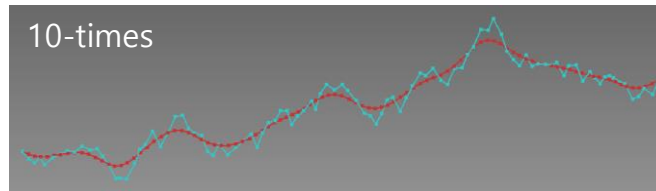
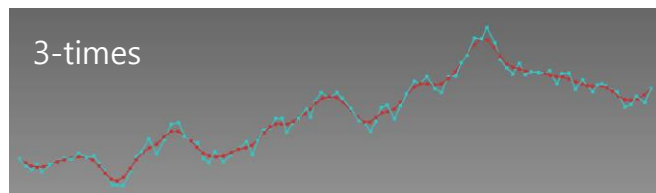
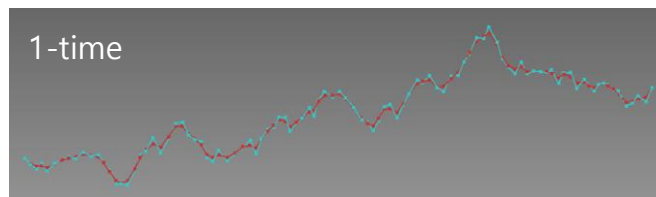
- Smoothing : 각 정점들의 각을 줄여 부드럽게 처리한다.
- 전/후 벡터 방향의 중간 벡터로 이동.



```
CGPolygon *iPolygon = const_cast<CGPolygon *>((const CGPolygon *)m_DPolygonsSmooth.GetPolygons());
VERIFY(iPolygon->Smooth(.5));
```



0.5 factor ↓



Polygons

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

Attributes

- Vertices, Separators
- Bounding Box, Center Point, Centeroid

Edition

- Compound, Explode
- Increase Density
- Reduce (Angle, Distance)
- Connect
- Open/Close
- Insert Polygons
- Remove in various ways
(Polygon/Polygons, Duplicated Point,
less than Number of Points)

Retrieve

- Is Only Vector

Weight & Measures

- Area
- Length

Transformation

- Vector, Vector2D
- Matrix2x2, Matrix2x3, Matrix3x3, Matrix2x4

Projection

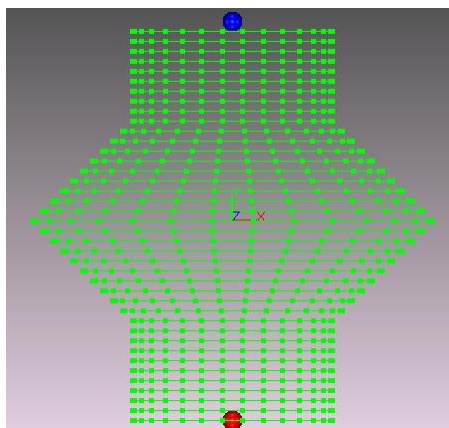
- Project Polygons Onto Mesh along a Direction
- Project Polygons Onto Mesh along Z-Axis

Polyhedron : Meshing

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

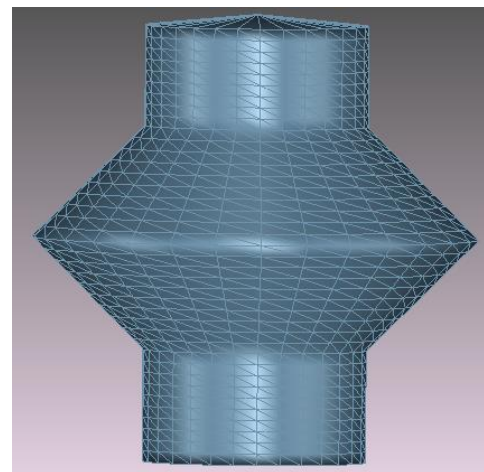
Meshing with Polygons

Create the circle line : Y축 방향으로 원점을 빨간 점부터 시작하여 원을 구역에 맞는 반지름으로 생성함.

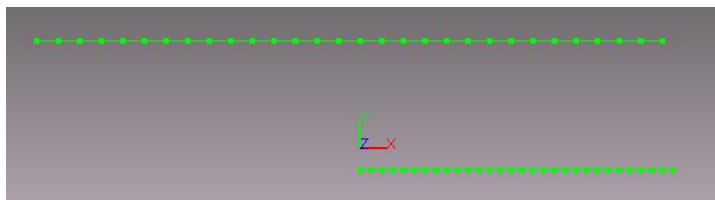


동일 점 개수를 가진 라인들

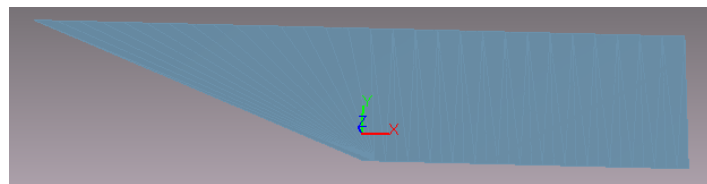
- 파란점 : Top Point
빨간점 : Bottom Point
영역 : 각 영역별로 10개의 라인
- Top 영역
 - Chamfer 영역
 - Cuff 영역
 - Bottom 영역



현재 Smooth wareframe 상태



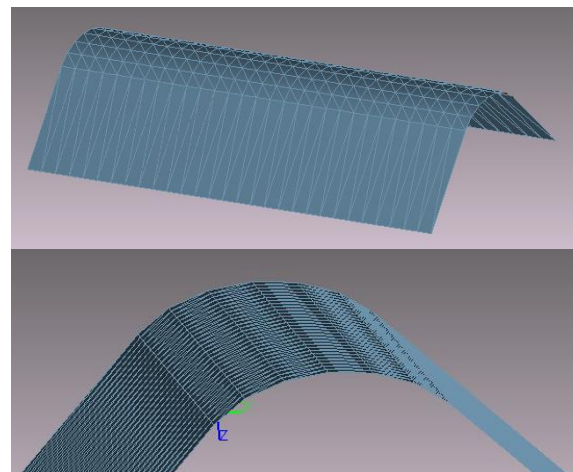
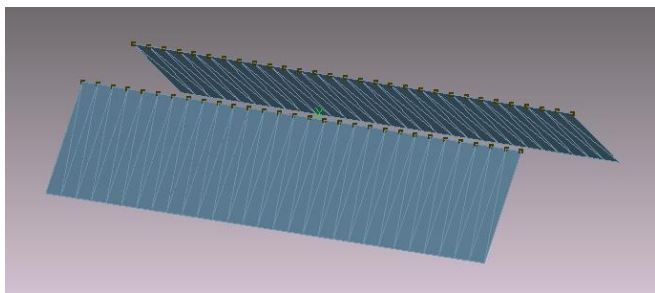
다른 점 개수를 가진 라인들



Polyhedron : Meshing

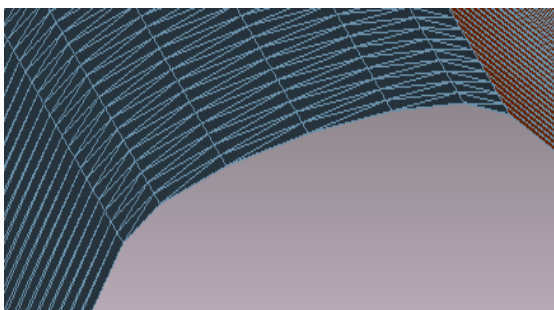
[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

Curved Meshing Between Meshes

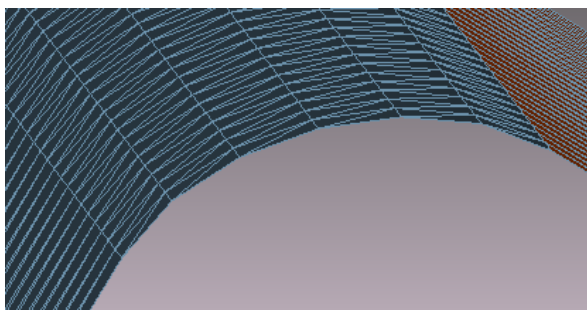


Number of intermediate points을 지정하고
Tension을 2로 공정 후 Create the mesh를 통해
Mesh가 잘 생성되었는지 확인한다.

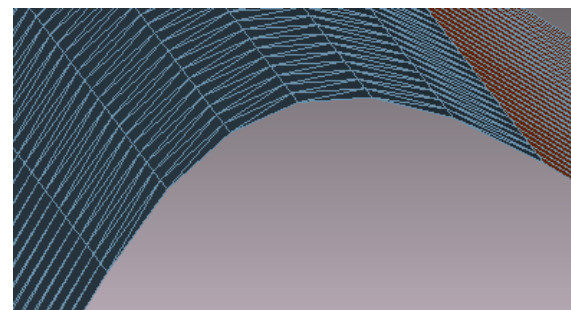
현재 Smooth wareframe 상태



Tension : 1



Tension : 2



Tension : 3

2D Polygon

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygon2D.h](#)

Attributes

- Vertices
- 2D Bounding Box, Center Point, Centeroid

Edition

- Offset
- Add/Remove Points
- Insert Points
- Add Polygon
- Open/Close
- Change Starting Point

Weight & Measures

- Area
- Length

Transformation

- Vector, Vector2D
- Matrix2x2, Matrix2x3, Matrix3x3, Matrix2x4

Set

- Union, Difference, Intersect,

Retrieve

- Retrieve Longest Point along a Direction
- Retrieve Closest Point along a Direction

2D Polygons

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

Attributes

- Vertices, Separators
- Bounding Box, Center Point, Centeroid

Edition

- Compound, Explode
- Offset
- Increase Density
- Reduce (Angle, Distance, Distance From Segment)
- Leave Out-line Only
- Clip Polygons by Rectangle/Polygon/Axis
- Cut Polygons with X or Y Axis Position
- Add Polygon/Polygons
- Insert Polygons
- Remove in various ways
(Polygon/Polygons, Duplicated Point,
less than Number of Points, Extreme Small Area)

Weight & Measures

- Length : mm

Transformation

- Vector, Vector2D
- Matrix2x2, Matrix2x3, Matrix3x3, Matrix2x4

Set

- Union, Difference, Intersect,

Fix

- Fix Winding

Projection

- Project Point along a Direction

2D Polygons

Edition

- Solid (4-Direction, 2-Direction)
- Chain
- Connect
- Fill Grid
- Twist
- Change Starting Point
- Invert (Vertex/Polygon reverse Order)

Creation

- Bitmap (Gray Scale). Close type
- Infill : Build Processor

Retrieve

- Orientation
- Is Point In Polygons
- Is Intersected two-Polygons
- Retrieve Longest Point/Polygon along a Direction
- Retrieve Lines For Jump lines
- Retrieve Intersected Point From Departure Point to Arrival Point

2D Polygons : Edition

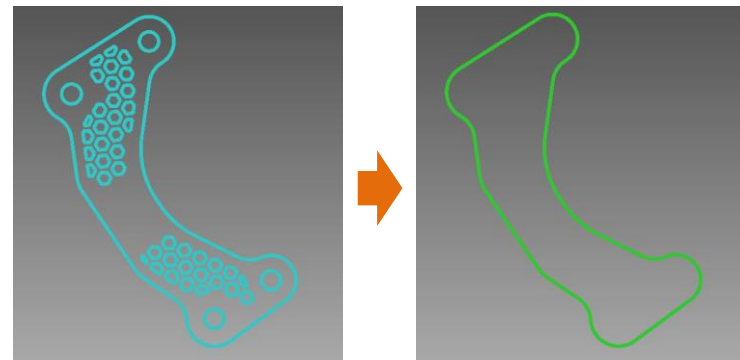
[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

Leave Outline Only

: 최외곽 라인만 남긴다.

```
CGPolygons2D * iPolygon = m_OriginPolygons.GetPolygons()->Clone();
ASSERT(iPolygon);

iPolygon->LeaveOutlineOnly();
m_Polygons1.SetPolygons(iPolygon);
```

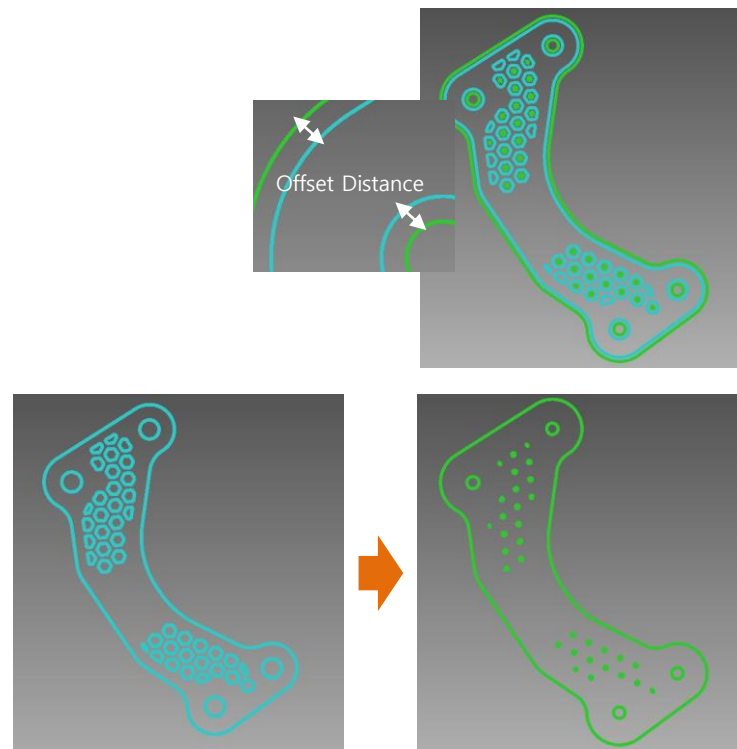


Offset

: 최외곽 라인 기준으로 임의의 거리만큼 늘려준다.

```
CGPolygons2D * iPolygon = m_OriginPolygons.GetPolygons()->Clone();
ASSERT(iPolygon);

iPolygon->Offset(1.);
m_Polygons1.SetPolygons(iPolygon);
```



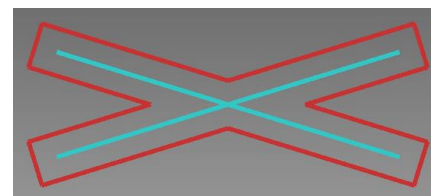
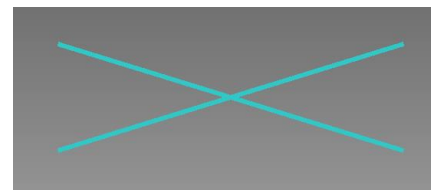
2D Polygons : Edition

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

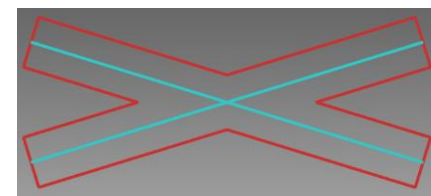
Solid

: 라인을 넓이가 있는 솔리드로 생성한다.

```
CGPolygons2D iPolygons2D;  
iPolygons2D.AddPolygon(CGLine(iLine1));  
iPolygons2D.AddPolygon(CGLine(iLine2));  
iPolygons2D.Solid(10., 0);  
m_ResultPolygons2D.SetPolygons(iPolygons2D.Clone());
```



4-Direction



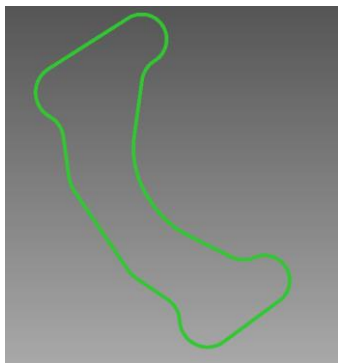
2-Direction

2D Polygons : Set

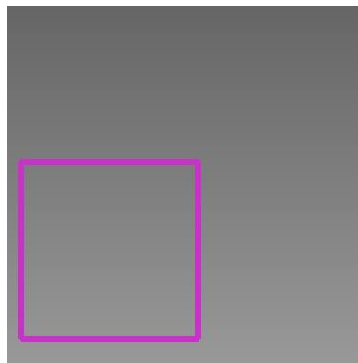
[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

Boolean

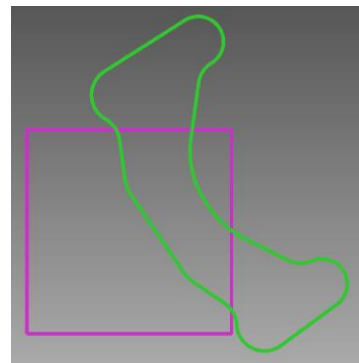
: 폴리곤에 대한 합/차/교집합을 수행한다.



Polygon1



Polygon2



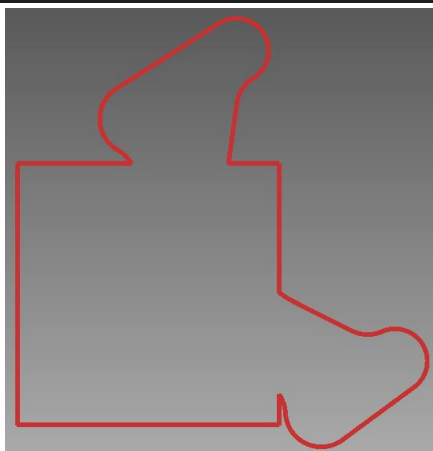
Polygon1 + Polygon2

```
CGPolygons2D iResult(*m_Polygons1.GetPolygons());  
iResult.AddPolygons(*m_Polygons2.GetPolygons());  
iResult.Union();
```

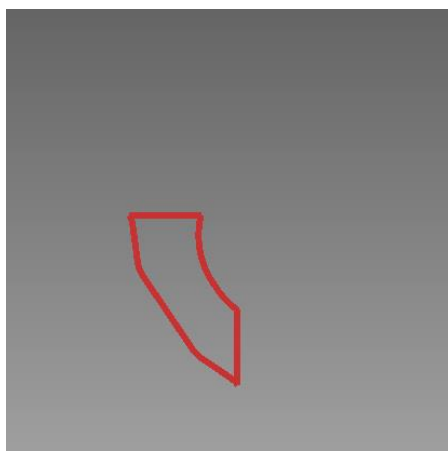
```
CGPolygons2D iResult(*m_Polygons1.GetPolygons());  
iResult.Intersect(*m_Polygons2.GetPolygons());
```

```
CGPolygons2D iResult(*m_Polygons1.GetPolygons());  
iResult.Difference(*m_Polygons2.GetPolygons());
```

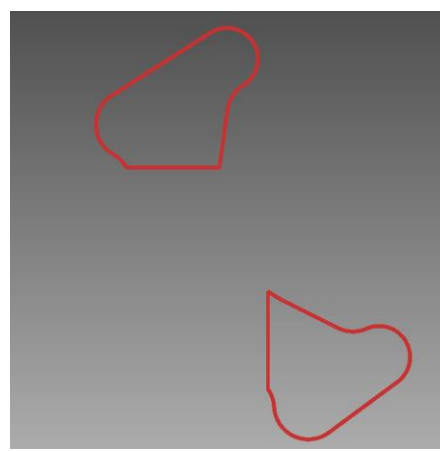
```
CGPolygons2D iResult(*m_Polygons2.GetPolygons());  
iResult.Difference(*m_Polygons1.GetPolygons());
```



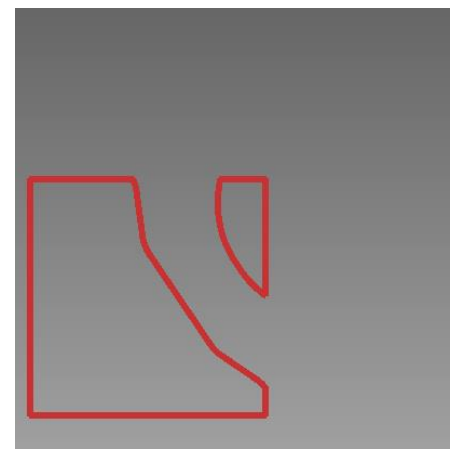
Polygon1 $\cup$ Polygon2



Polygon1 $\cap$ Polygon2



Polygon1 - Polygon2

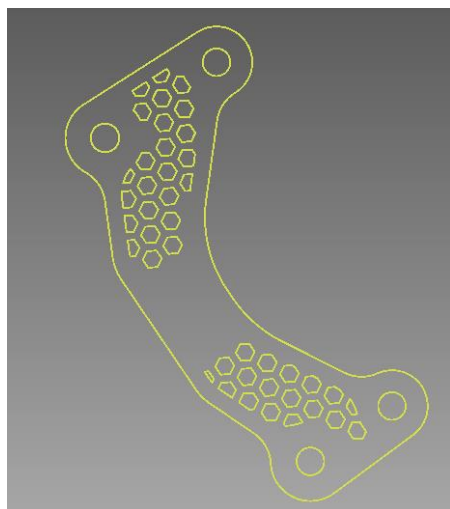


Polygon2 - Polygon1

2D Polygons : Creation

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

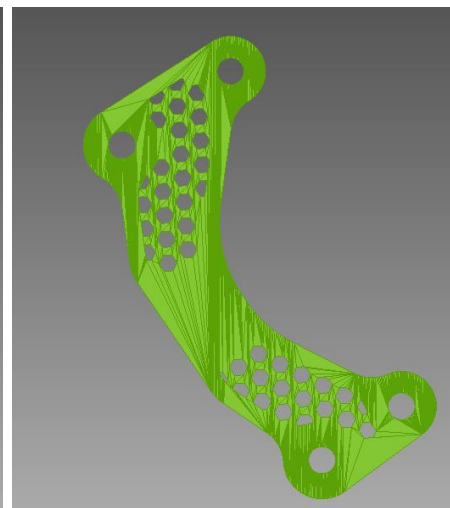
Meshing in 2D Polygons



2D Polygons



Plat Representation



Plat Wire Representation

Meshing from TEXT

Font Name : Gulim Regular

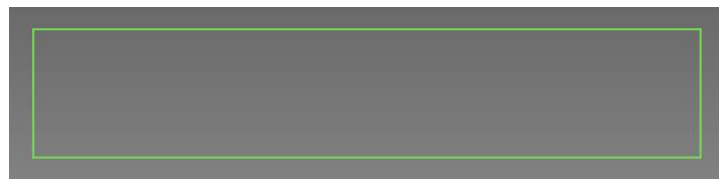
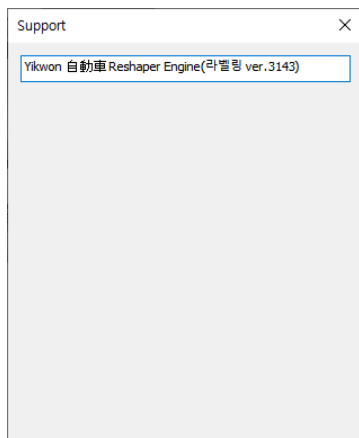


2D Polygons : Creation

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

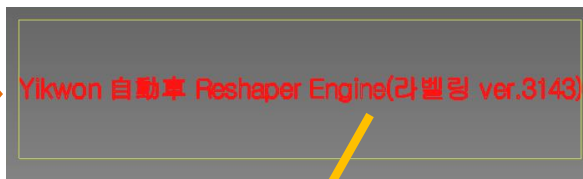
Text Polygons

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GText.h](#)



텍스트가 그려질 영역을 드래그 한다.

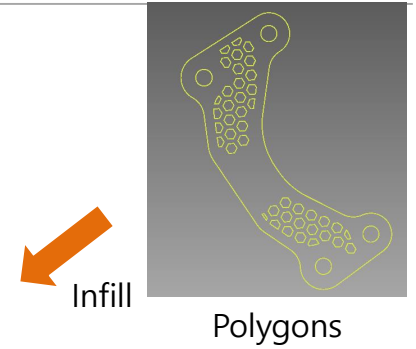
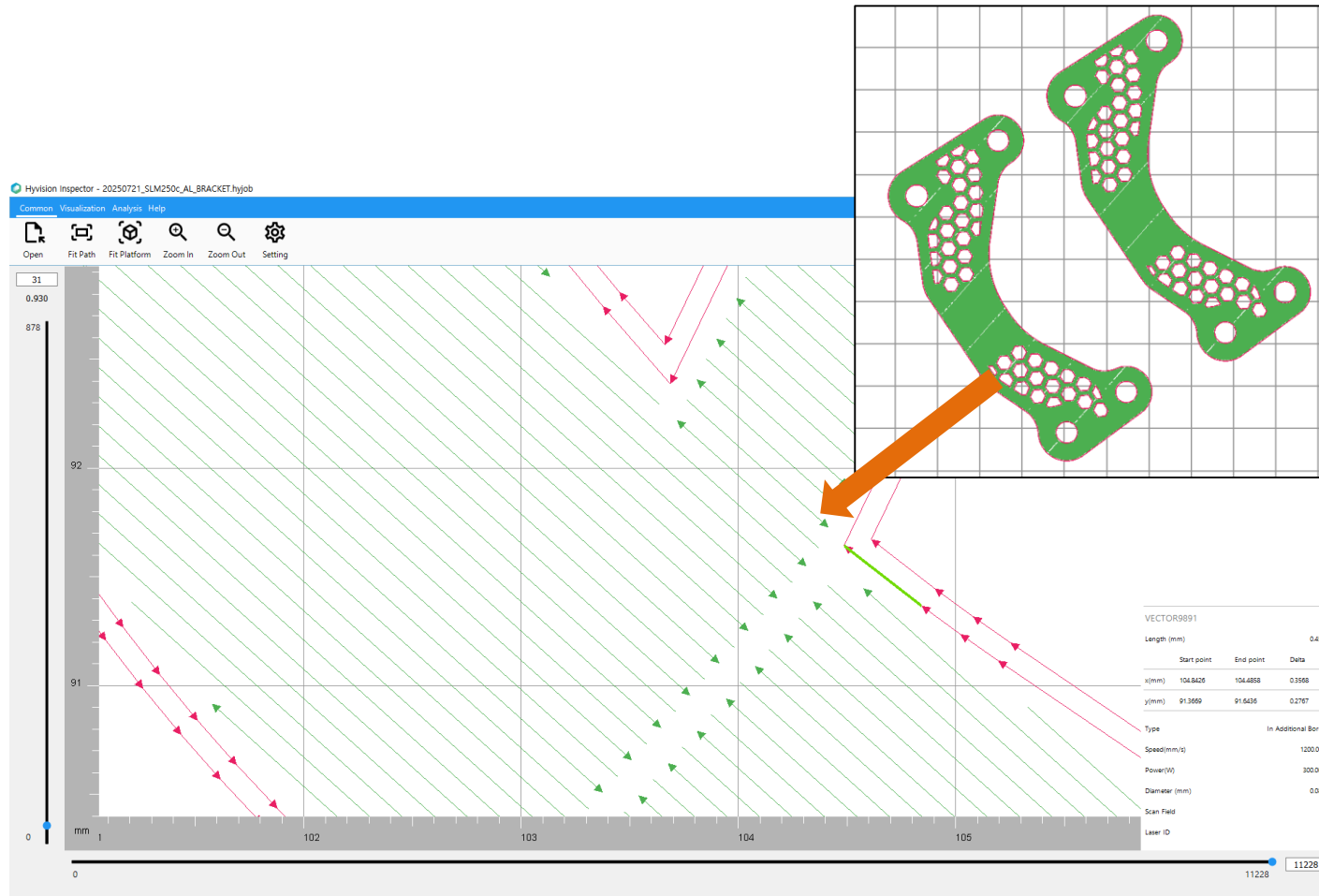
Font Name : Gulim Regular



2D Polygons : Creation

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPolygons2D.h](#)

Infill(Build Processor) : Hatch/Border Vectors



Line

Attributes

- Departure, Arrival
- Bounding Box, Center Point

Edition

- Invert (Departure, Arrival)

Retrieve

- Retrieve Distance from Point
- Retrieve a Closest Polygon's Point from Line
- Is On Line

Weight & Measures

- Distance

Transformation

- Vector
- Matrix3x3, Matrix3x4

Projection

- Project Point On Line
- Project Point along a Direction
- Project Point On Plane

Intersection

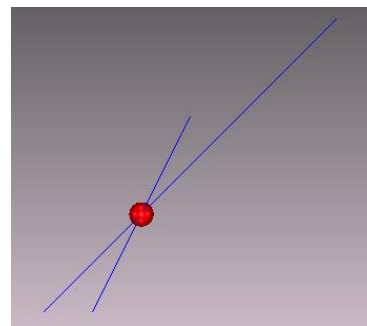
- Retrieve Intersected Point crossed two-lines

Intersection

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GLine.h](#)

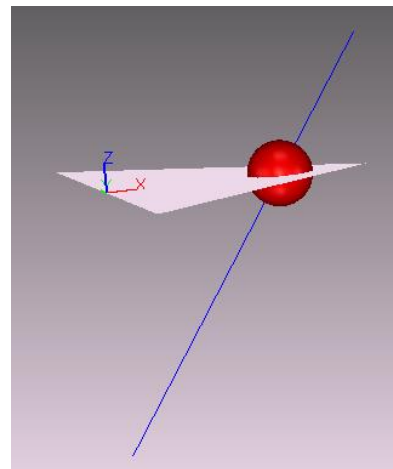
Intersect the two-lines

- Smoothing : 전/후 Vertex의 각을 줄이는 기능



2개의 라인을 교차하는 점을 확인한다.

Intersect with Ray on Triangle



삼각형 상에서 레이가 통과하는 교차점을 확인한다.

2D Line

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GLine2D.h](#)

Attributes

- Departure, Arrival
- Bounding Box, Center Point

Edition

- Invert (Departure, Arrival)
- Cut Outside By X Boundary

Retrieve

- Retrieve the Slope of line
- Retrieve Solid (Thickness) of line
- Is Point/Line On Line

Weight & Measures

- Distance

Transformation

- Vector2D
- Matrix2x2, Matrix2x3

Projection

- Project Point on Line along a Direction

Intersection

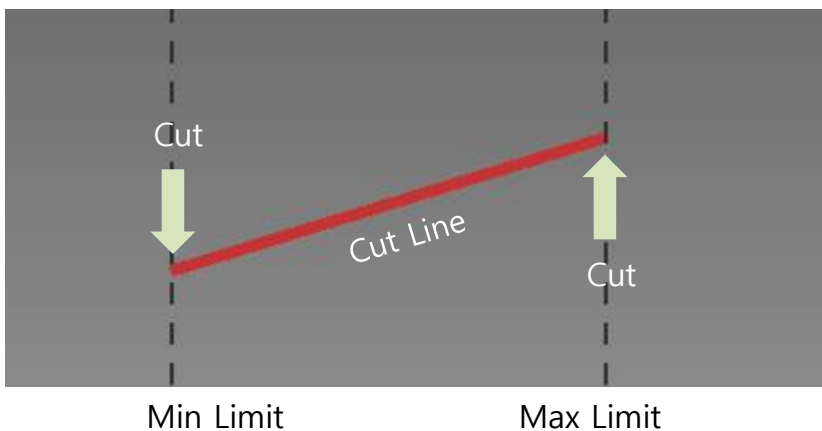
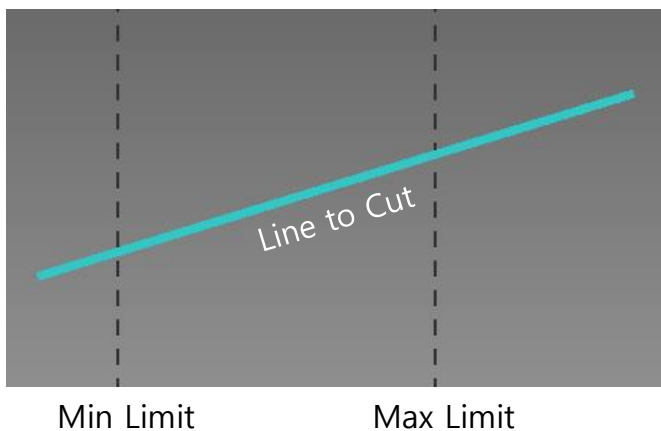
- Retrieve Intersected Point crossed two-lines
- Retrieve Intersected Point crossed X or Y-Axis

2D Line : Edition

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GLine2D.h](#)

Cut Outside by x boundary

- X의 Min/Max 제한 영역 밖은 잘라낸다.



```
CGLine2D iLine = m_DLine.GetLine();

DOUBLE iDepartureX = m_MinLimit.GetLine().First().GetX(), iDepartureY = 0.,
        iArrivalX = m_MaxLimit.GetLine().First().GetX(), iArrivalY = 0.;

iLine.CutLineAtX(iDepartureX, iDepartureY);
iLine.CutLineAtX(iArrivalX, iArrivalY);
m_Result.Init(CGPointDouble(iDepartureX, iDepartureY), CGPointDouble(iArrivalX, iArrivalY));
```

Triangle

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GTriangle.h](#)

Attributes

- Point 1~3
- Bounding Box, Center Point

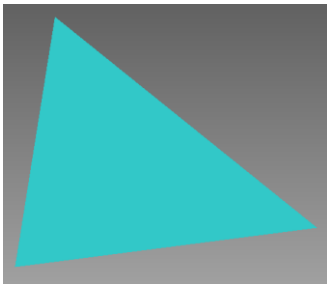
Edition

- Invert Winding

Intersection

- Retrieve two Intersected Points On the Triangle's Plane
- Retrieve Intersected Point On Triangle
- Retrieve Intersected Line By Z Height

Shape & Code



```
CGTriangle iTriangle(CGPointDouble(50, 25, 0), CGPointDouble(150, 40, 0), CGPointDouble(70, 125, 0));
m_DTriangle[0].SetTriangle(iTriangle);
```

Weight & Measures

- Area
- Circumference

Transformation

- Vector
- Matrix2x2, Matrix2x3, Matrix3x3, Matrix3x4

Projection

- Project Point On Plane (along a Direction)

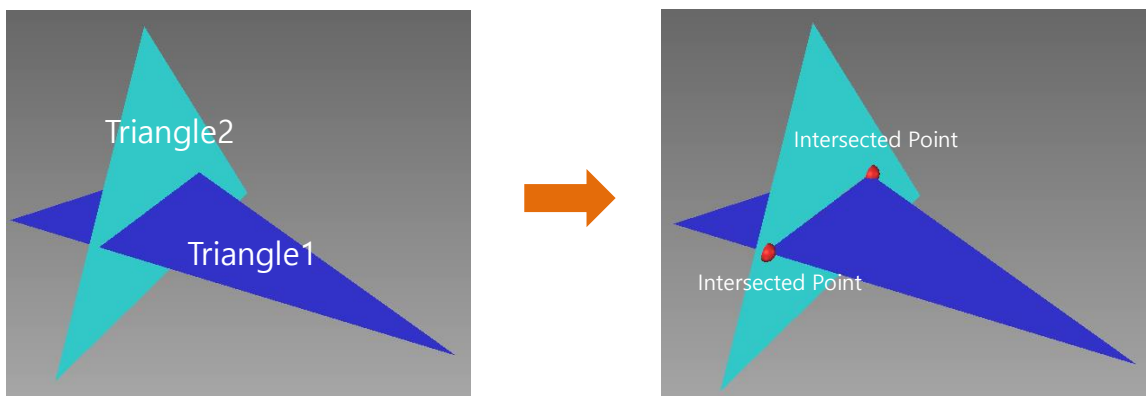
Retrieve

- Retrieve Normal
- Retrieve Edge Vector
- Is Share Vertex Two-Triangles
- Is Vertex In Triangle
- Is Vertex On Edge
- Is On Triangle Plane

Triangle : Intersection

Two Intersected Points on the Two Triangle's Planes

- 2개의 각 삼각형 평면에 교차하는 점을 찾는다



Two Triangles

```
const CGTriangle & iTriangle1 = m_DTriangle[0].GetTriangle();
const CGTriangle & iTriangle2 = m_DTriangle[1].GetTriangle();

CGPointDouble iIntersectedPoint[2];
CGTriangle::Intersect2(iTriangle1.GetPoint(0), iTriangle1.GetPoint(1), iTriangle1.GetPoint(2),
    iTriangle2.GetPoint(0), iTriangle2.GetPoint(1), iTriangle2.GetPoint(2),
    iIntersectedPoint[0], iIntersectedPoint[1]);

m_DInterSectedPoint[0].SetPoint(iIntersectedPoint[0]);
m_DInterSectedPoint[1].SetPoint(iIntersectedPoint[1]);
```

Plane

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPlane.h](#)

Attributes

- A, B, C, D

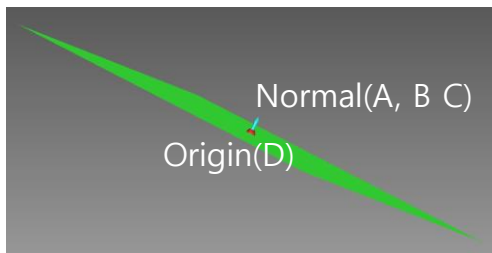
Edition

- Invert

Retrieve

- Retrieve Normal
- Retrieve X, Y Position By U/V

Shape & Code



```
const CGPlane iPlane = iCircle.GetPlane(iDegree);
const CGPointDouble iOriginPlane = iPlane.Interpolate(0, 0);
```

Weight & Measures

- Area
- Circumference

Transformation

- Vector
- Matrix3x3, Matrix3x4

Projection

- Project Point On Plane (along a Direction)

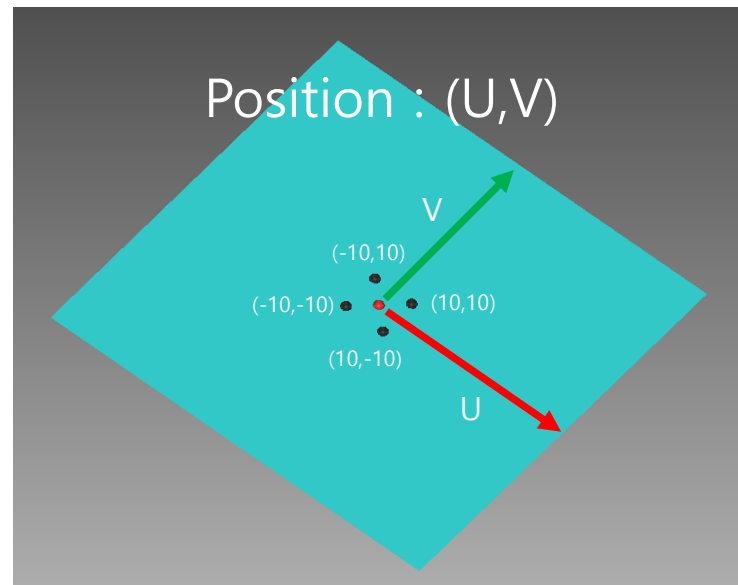
Circle : Retrieve

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GPlane.h](#)

Retrieve X,Y Position By UV

- 평면에서 U, V 위치의 포지션 데이터를 반환한다.

```
const DOUBLE iDegree      = TO_RADIAN(60);  
const CGCircle & iCircle  = m_DCircle.GetCircle();  
const CGPlane iPlane     = iCircle.GetPlane(iDegree);  
const CGPointDouble iPoint1 = iPlane.Interpolate(-10, -10);  
const CGPointDouble iPoint2 = iPlane.Interpolate(10, -10);  
const CGPointDouble iPoint3 = iPlane.Interpolate(10, 10);  
const CGPointDouble iPoint4 = iPlane.Interpolate(-10, 10);
```



Circle

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GCircle.h](#)

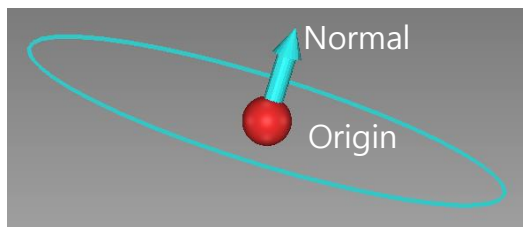
Attributes

- Center Point, Direction, Radius

Edition

- Invert Normal

Shape & Code



```
CGCircle iCircle(CGPointDouble(50, 50, 0), 25, CVector::ZAxis());
m_DCircle.SetCircle(iCircle);
```

Weight & Measures

- Area
- Circumference

Transformation

- Vector

Creation

- Create Circle Lines

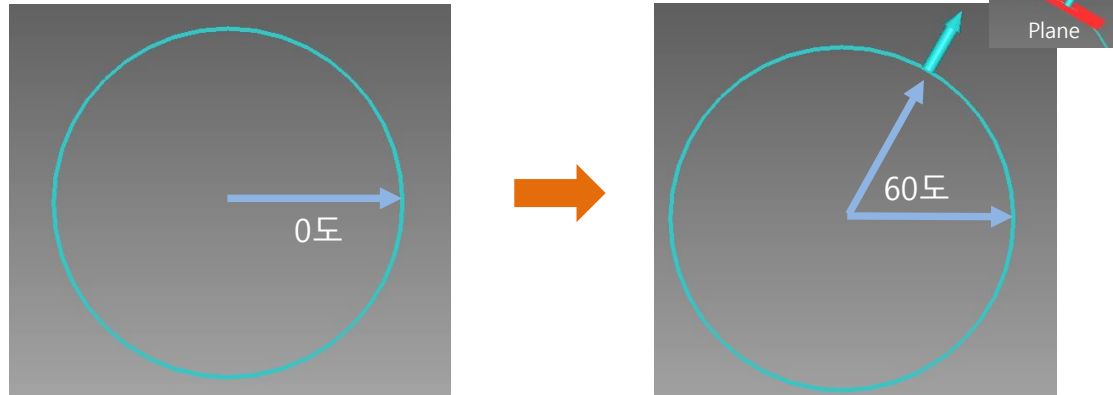
Retrieve

- Retrieve a Point at Angle
- Retrieve a Plane of Tangent at Angle
- Retrieve Slope(Vector) of Tangent at Angle

Circle : Retrieve

Retrieve a Plane of Tangent at Angle

- 원의 시작 각도 기준에서 임의의 각도의 평면을 찾는다



Two Triangles

```
const DOUBLE iDegree          = TO_RADIAN(60);
const CGCircle & iCircle      = m_DCircle.GetCircle();
const CGPlane iPlane         = iCircle.GetPlane(iDegree);
const CGPointDouble iTangentPoint = iCircle.GetPoint(iDegree);
const CGPointDouble iOriginPlane = iPlane.Interpolate(0, 0);

const CVector iDirectionToMove(iOriginPlane, iTangentPoint);
const CGPointDouble iPoint1      = iDirectionToMove + iPlane.Interpolate(-10, -10);
const CGPointDouble iPoint2      = iDirectionToMove + iPlane.Interpolate(10, -10);
const CGPointDouble iPoint3      = iDirectionToMove + iPlane.Interpolate(10, 10);
const CGPointDouble iPoint4      = iDirectionToMove + iPlane.Interpolate(-10, 10);
```

2D Circle

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GCircle2D.h](#)

Attributes

- Center Point, Radius

Retrieve

- Retrieve Point at Angle

Weight & Measures

- Area
- Circumference

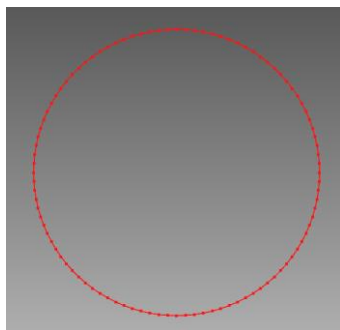
Transformation

- Vector2D

Creation

- Create Circle Lines

Shape & Code



```
CGCircle2D iCircle(CGPoint2DDouble(50, 50), 10);  
  
vector<Coordinate2D> oCircleLine;  
iCircle.ToMultiLine(oCircleLine, 100);  
CGPolygons2D * iPolygons2D = new CGPolygons2D(oCircleLine);
```

Cloud

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GCloud.h](#)

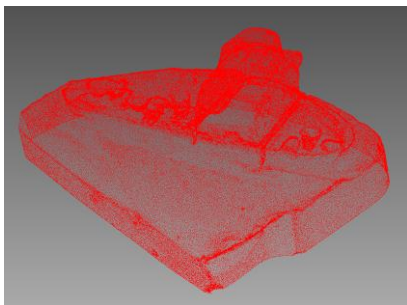
Attributes

- Points
- Bounding Box, Center Point, Centeroid

Edition

- Invert

Shape & Code



```
CGCloud iCloud;  
iCloud.SetPoints(iMesh->GetVertices());  
CDCloud * iDCloud = new CDCloud(iCloud);
```

Convex Hull

- Retrieve Polygon for Convex Hull

Transformation

- Vector
- Matrix3x3, Matrix3x4

Retrieve

- Retrieve Longest Distance from a Point
- Retrieve Longest Point along a Direction

Projection

- Project Points Onto Plane (Point, Normal)

2D Cloud

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GCloud2D.h](#)

Attributes

- Points
- Bounding Box, Center Point, Centeroid

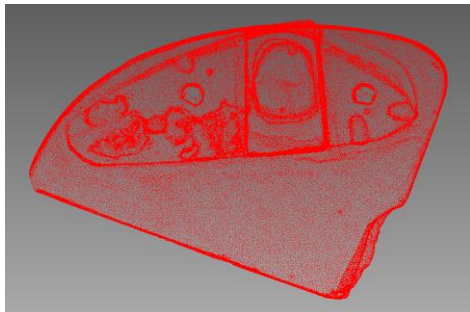
Edition

- Invert

Retrieve

- Retrieve Longest Distance from a Point
- Retrieve Longest Point along a Direction

Shape & Code



```
CGCloud2D iCloud;  
iCloud.SetPoints(iPoint2Ds);
```

Transformation

- Vector
- Matrix2x2, Matrix2x3

Hexahedron

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GHexahedron.h](#)

Attributes

- Rectangle, Direction, Height
- Bounding Box, Center Point

Edition

- Invert (Direction)

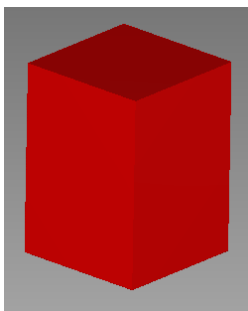
Weight & Measures

- Volume

Transformation

- Vector
- Matrix3x3, Matrix3x4

Shape & Code



```
CGRectangle iRectangle(CGPointDouble(20, 20, 20),
    CGPointDouble(30, 30, 20),
    CGPointDouble(20, 40, 20),
    CGPointDouble(10, 30, 20));
CGHexahedron iHexahedron(iRectangle, CVector::ZAxis(), 20);
```

Ellipse

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GCircle.h](#)

Attributes

- Origin, Direction, Horizontal/Vertical Radius

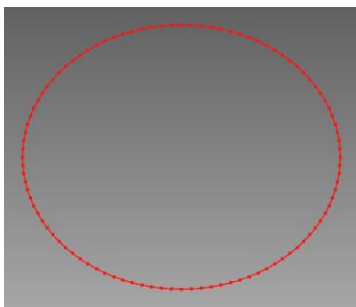
Edition

- Invert Normal

Retrieve

- Retrieve Polygon/Polygon2D

Shape & Code



```
CGEllipse iEllipse(CGPointDouble(50, 50, 20), CVector::ZAxis(), 12, 10);

CGMultiLine oEllipseLine;
iEllipse.ToMultiLine(oEllipseLine, 100);
CGPolygons2D * iPolygons2D = new CGPolygons2D(oEllipseLine);
```

Weight & Measures

- Area
- Circumference

Transformation

- Vector

Creation

- Create Circle Lines
- Create 2D Circle Lines

2D Ellipse

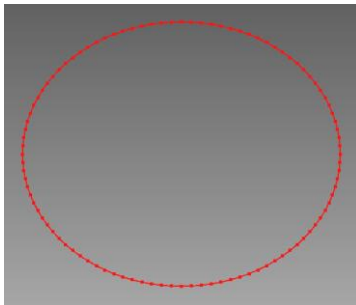
Attributes

- Origin, Horizontal/Vertical Radius

Retrieve

- Retrieve Polygon/Polygon2D

Shape & Code



```
CGEllipse2D iEllipse(CGPoint2DDouble(50, 50), 12, 10);  
  
vector<Coordinate2D> oEllipseLine;  
iEllipse.ToMultiLine(oEllipseLine, 100);  
CGPolygons2D * iPolygons2D = new CGPolygons2D(oEllipseLine);
```

Weight & Measures

- Area
- Circumference

Transformation

- Vector2D

Creation

- Create 2D Circle Lines

Cylinder

Attributes

- Origin, Direction, Height, Radius

Edition

- Invert (Horizontal<->Vertical)

Retrieve

- Retrieve a Circle By Z-Axis Position

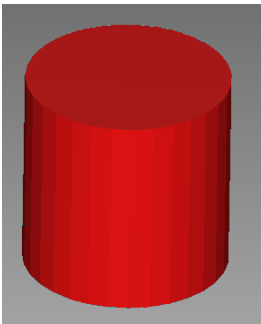
Weight & Measures

- Volume

Transformation

- Vector

Shape & Code



```
CGCylinder iCylinder(CGPoint2DDouble(50, 50), CVector::ZAxis(), 10, 20);  
  
CDCylinder * iDCylinder = new CDCylinder(iCylinder);  
iDCylinder->SetColor(RGB(255, 20, 20));
```

2D Cross

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GCross2D.h](#)

Attributes

- Origin, Horizontal/Vertical Length, Angle

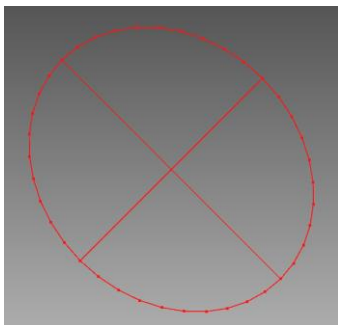
Edition

- Invert (Horizontal <-> Vertical)

Retrieve

- Retrieve Polygons For Rectangle/Ellipse Type (include Boundary)

Shape & Code



```
CGCross2D iCross(CGPoint2DDouble(50, 50), 12, 10, TO_RADIAN(45));

CGPolygons2D * iPolygons2D = new CGPolygons2D;
iCross.Get(*iPolygons2D, CGCross2D::BOUNDARYTYPE_ELLIPSE);
```

Transformation

- Vector

2D Spiral

[3DReshaperSDK/Reshaper3DEngine/Kernel/Object/Geometry/GSpiral2D.h](#)

Attributes

- Center Point, Inner/Outer Radius, Offset Distance

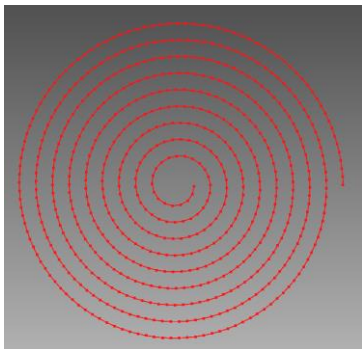
Edition

- Invert (Horizontal <-> Vertical)

Retrieve

- Retrieve Polygons For Spiral (various ways)

Shape & Code



```
CGSpiral2D iSpiral(CGPoint2DDouble(50, 50), 1, 10, 1);  
  
CGPolygons2D * iPolygons2D = new CGPolygons2D;  
iSpiral.ToPolygons2D(0.5, *iPolygons2D);
```

Transformation

- Vector2D

Files

Project Files

Read : rsp3d

write : rsp3d

3D Part Files

Read : stl(ascii, bin)

write : stl(bin)

Slice Files

Read : slc, cli, 2dpolygons

Write : 2dpolygons

Printing Files

Read : job, hyslc, hyjob

write : hyslc, hyjob,

Log Files

Write : log

etc Files

Read : xml, ini

Write : xml, ini

Thank you